CHECKLIST · CLAUDE CODE · GITHUB · CLOUDFLARE PAGES

Website cheat sheet

Checklist for building a website with Claude Code, GitHub and Cloudflare Pages. Nine phases, all repeated for every website; phase 9 only when the site sells something. Tick the boxes as you go.

PER WEBSITE

Run phases 1 to 8 for every website. Phase 9 only when the site sells something.

ONE CHAT

Keep one Claude Code chat per website until it is full. Ask for a handover before a new one.

ONE BRANCH

Everything on **main** . Every push goes live on Cloudflare within minutes.

Phase 1: GitHub repository

1. github.com → New repository → name it after the website → Private → tick "Add a README" → Create.
2. Check that the default branch is `main` (Settings → General).
3. One repository per website. Nothing else to install or connect.

Phase 2: Start a Claude Code chat

1. claude.ai/code → New session → choose the repository → branch `main` .
2. First message, in this order:
   - What the website is for and who it is for.
   - Languages.
   - Pages it needs.
   - If it must look like an earlier website: name that repository, so Claude can copy its structure.
   - "Work on `main` only. Commit and push after every step."
3. Then paste the information from Phase 3, one message per topic.
4. Keep the same chat for as long as it works. Start a new one only when the chat is full.
5. Before starting a new chat: ask Claude for a handover (what is done, what is open, where things live). Paste it as the first message of the new chat.
6. If the chat worked on its own branch (`claude/...`): ask it to merge into `main` and delete that branch.

Phase 3: Information to give

Collect everything before the chat starts. What is missing becomes a placeholder.

- [ ] One or more reference websites for the look and feel. Fonts and colours come from them unless you say otherwise
- [ ] Colours as hex codes, only if they must differ from the reference
- [ ] Font names, only if they must differ from the reference
- [ ] Logo as SVG or PNG, if there is one
- [ ] Pages and their order (home, about, offer, agenda, blog, contact, ...)
- [ ] All texts, per language, as a Pages or Word file; say which language to translate from
- [ ] Photos: a shared folder (Google Drive, "anyone with the link"); say which ones not to use
- [ ] Events or products: date, times, price, maximum number, short description
- [ ] Contact details: name, email, phone, address, VAT number
- [ ] Legal: terms, cancellation rules, privacy text if a lawyer gave one
- [ ] Links: other websites of the same owner, social media
- [ ] Domain name: existing (where is it registered?) or to buy
- [ ] Email: where is the mailbox hosted?

Phase 4: Put it online on Cloudflare Pages

Do this as soon as the first version exists. The `.pages.dev` address is your preview; every push updates it within minutes.

1. dash.cloudflare.com → sign up or log in. One account for all sites and domains.
2. Workers & Pages → Create → Pages → Connect to Git → authorise GitHub → pick the repository.
3. Build settings. Ask Claude for the exact values and paste them. Typical:
   - Production branch: `main`
   - Framework preset: the site's framework, or None
   - Build command: `npm run build`
   - Build output directory: `dist`
   - Environment variable: `NODE_VERSION` = `22`
4. Save and Deploy. After a few minutes you get a `.pages.dev` address.
5. Forms: Cloudflare Pages has no built-in form handling. Create a free Web3Forms or Formspree account → copy the key → ask Claude to connect the forms to it.
6. Not updating after a push? Deployments tab → open the red build → paste the error in the chat.

Phase 5: Review on the live site

1. Open the `.pages.dev` address on a laptop and on a phone.
2. Check in this order: texts, photos and their placement, menu and links, language switch, forms, phone view.
3. Feedback as one list per round in the chat. Claude fixes and pushes; the site updates within minutes.
4. Ask for a brand guide: a hidden page on the site (`/brand/`) with the colours and their hex codes, fonts, buttons, labels, photo style and tone of voice; a short brand story; and a standalone `brand.css` with the same colours and fonts. Use it for social media, documents, presentations and the next website.
5. Ask for a README and a handover file: open points, how to add content.
6. Stop when the owner has read every page once on the live address.

Phase 6: Domain and email

A. Domain registered elsewhere

1. Cloudflare → Add a domain → type the domain → Free plan → Continue.
2. Check the imported records before continuing: MX (mail), TXT (SPF, verifications), A (current site).
3. Copy the two Cloudflare nameservers shown.
4. At the current registrar: domain settings → Nameservers → replace both with the Cloudflare ones → Save.
5. Wait for the Cloudflare email "domain is active" (minutes to 24 hours). Old site and email keep working.

B. New domain: Cloudflare → Domain Registration → Register → search → buy. Turn on auto-renew.

C. Connect the domain to the site

1. Workers & Pages → the project → Custom domains → Set up a custom domain → the domain → Activate.
2. Repeat for `www.` or for a subdomain if the site lives next to another one.
3. Cloudflare writes the records and the certificate. Live within minutes.
4. Tell Claude the final address so the site's configuration carries it → push.

D. Email. Mail keeps working as long as the MX records stay unchanged. Moving the mailbox = new MX records from the new provider, entered in Cloudflare. Never cancel the old host before the mailbox has moved and worked for a week.

E. Optional, later: transfer the registration to Cloudflare. Old registrar → unlock the domain → request the transfer (EPP) code → Cloudflare → Transfer domains → paste the code. Up to five days, nothing goes offline.

Phase 7: Testing

On the real domain, on a laptop and on a phone, in a browser where you are not logged in anywhere.

- `https://domain` , `https://www.domain` and every language open with a padlock
- Every menu item and footer link opens a page, no 404
- The language switch lands on the same page in the other language

- Phone: menu opens and closes, headline fits, photos load
- Every form: send a test, the email arrives, the thank-you page shows
- Detail pages (events, products): dates, times, prices correct, buttons go to the right place
- Map shows the right spot
- The old website is gone
- Google Search Console: add the domain, verify via the Cloudflare DNS record
- The owner confirms in writing that the site may go public

Anything wrong: one list in the same chat.

Phase 8: Afterwards

CHANGE	HOW	LIVE AFTER
New event, article, product	Copy an existing content file, fill it in; or ask in the chat	minutes
New photo	Upload to the images folder on github.com, tell Claude where it goes	minutes
Text change	One sentence in the chat; or edit the file on github.com	minutes
Colour or font	Ask Claude; it lives in the site's theme file	minutes

Logins to keep in a password manager: GitHub, Cloudflare, registrar, mail provider, form service, payment provider.

Yearly: domain renewal (auto-renew on), check that the owner's email still works, check the form service.

Running costs: GitHub free, Cloudflare Pages free, forms free, domain about 10 to 20 euro a year.

Phase 9: Payments (Stripe + Billit)

Only when the site sells tickets or products. Stripe takes the payment and emails the receipt and invoice to private customers. Billit makes the Belgian invoices for companies (Peppol) and feeds the accountant. Ask the accountant first whether the service carries 21 % VAT or is exempt.

A. Stripe

1. stripe.com → create an account with the owner's company details (VAT number, IBAN). Verification takes a day or two.
2. Settings → Payment methods → switch on Bancontact, cards, Apple Pay, Google Pay, iDEAL.
3. Settings → Branding → logo and colour, so the payment page matches the site.
4. Settings → Tax → switch on Stripe Tax → add the Belgian VAT registration → set the tax category.
5. Settings → Emails → switch on receipts to customers. Settings → Invoices → numbering prefix, footer with VAT number, address and terms.
6. Product catalogue → Add product per ticket or product: name, price, VAT behaviour.

7. Payment Links → New → pick the product → collect name, email and phone → add custom fields if needed → switch on "Collect tax ID" for business customers → switch on "Create an invoice" → after payment: redirect to the site's thank-you page.
8. Copy the payment link and give it to Claude: "add this as the Buy ticket button on event X".
9. Test in Test mode first (toggle top right, test card 4242 4242 4242 4242), then switch to Live and make the links again.
10. Settings → Payouts → bank account → daily or weekly.

B. Billit

1. billit.be → create an account with the company details → choose a plan.
2. Settings → logo, invoice numbering, VAT rate, payment terms, bank account.
3. Peppol → register the company's Peppol address (Billit is an access point). Needed anyway to receive suppliers' invoices.
4. Give the accountant access (Settings → Accountant).
5. Optional: link the bank account so payments are matched automatically.
6. Business customer paid through Stripe (tax ID filled in) → make the invoice in Billit for that company → send via Peppol → mark as paid.
7. Private customers: Stripe's receipt and invoice are enough. Monthly: Stripe → Reports → export payouts for the accountant.
8. Optional, later: automate Stripe → Billit with Make or Zapier, one Billit invoice per Stripe payment.

C. On the website

1. Claude adds a Buy ticket button per event or product with the Stripe link.
2. Price, VAT, cancellation and refund terms stand on the page above the button. Required by law.
3. The registration or contact form stays for free options and questions.

D. Test one real payment of a small amount

- [ ] Bancontact works on a phone
- [ ] Receipt email and invoice PDF arrive, with correct VAT and company details
- [ ] Payment shows in Stripe, redirect lands on the thank-you page
- [ ] A test with a VAT number ends in a Billit invoice sent via Peppol
- [ ] Refund the test payment from Stripe